
PROJECT REPORT

RetailMart's fragmented data challenges

*A Production-Grade Medallion Architecture Data Engineering Pipeline for a RetailMart
Centralized Data Platform*

Submitted by

Raj Kumar

B.Tech, Computer Science (AI & Data Science)
Poornima University, Jaipur

GitHub Repository: <https://github.com/Raj-Kumar-Sharma/RetailMart-Final-Project>

Table of Contents

Table of Contents	2
1. Abstract.....	3
2. Introduction	3
2.1 Problem Statement	3
2.2 Project Objective	3
2.3 Scope.....	3
3. Technology Stack	Error! Bookmark not defined.
4. System Architecture	Error! Bookmark not defined.
4.1 The Medallion Architecture	Error! Bookmark not defined.
5. Data Model.....	Error! Bookmark not defined.
6. Bronze Layer —	Error! Bookmark not defined.
7. Silver Layer — Validation & Trust	Error! Bookmark not defined.
7.1 Data Quality Engine	Error! Bookmark not defined.
7.2 De-duplication and Derived Columns	Error! Bookmark not defined.
8. Gold Layer — Business Intelligence	Error! Bookmark not defined.
9. Pipeline Orchestration.....	Error! Bookmark not defined.
11. Executive Dashboard	Error! Bookmark not defined.
13. Challenges Faced.....	Error! Bookmark not defined.
14. Software Engineering Principles	Error! Bookmark not defined.
16. Future Scope.....	Error! Bookmark not defined.
17. Application Screenshots	Error! Bookmark not defined.
18. Conclusion.....	Error! Bookmark not defined.
19. References.....	Error! Bookmark not defined.

(Right-click the table above and choose “Update Field” in Microsoft Word to populate page numbers.)

1. Abstract

The **RetailMart Enterprise Data Platform** is an end-to-end data engineering solution designed to process, govern, and analyze large-scale retail transaction datasets. The project implements a production-style Medallion Architecture (\$Bronze \rightarrow Silver \rightarrow Gold\$) within the **Databricks Free Edition** cloud ecosystem.

Leveraging **PySpark** and **Delta Tables**, the pipeline transforms raw operational retail data (Orders, Customers, Products, Order Items, and Payments) into structured, highly reliable data assets. The platform features an automated pipeline orchestrating multiple processing notebooks, data cleaning configurations, and business logic enforcement. Final data layers are exposed to **SQL Analytics** and materialized into highly interactive executive dashboards using both **Databricks Dashboards** and **Power BI**, providing actionable insights on key metrics such as total revenue, top-performing product categories, consumer segmentation, and logistics fulfillment.

2. Introduction

2.1 Problem Statement

Modern retail operations generate vast quantities of transactional and relational data distributed across fragmented, unvalidated sources. This operational data often suffers from quality defects including duplicate entry flags, null-heavy fields, structural anomalies, inconsistent formatting, and orphaned keys. Turning this raw operational footprint into trustworthy business metrics requires a robust, scalable pipeline that dynamically cleans records, preserves operational histories, and exposes refined analytical formats without corrupting downstream reporting.

2.2 Project Objective

The central objective of the RetailMart project is to build and deploy a unified, enterprise-grade data platform using PySpark inside Databricks. By systematically engineering a layered Medallion data workflow, the pipeline aims to ingest raw historical files, execute advanced transformation rules, enforce business constraints, and export reliable analytics matrices into Power BI to enable data-backed strategic decisions.

2.3 Scope

- Ingestion of raw retail source tables across major business entities.
- Multi-stage pipeline processing built completely on PySpark and Delta Lake.
- Strict Silver-layer validations for data profiling, normalization, and deduplication.
- Gold-layer star-schema aggregation for optimized relational metrics.
- Implementation of advanced SQL analysis patterns directly inside Databricks.
- Cross-platform reporting via native Databricks UI and dedicated Power BI interfaces.

3. Technology Stack

The platform architecture is built heavily upon cloud data warehousing practices and scalable cluster processing frameworks.

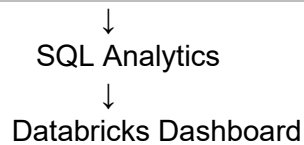
Category	Technology	Language / Interface
Data Processing	PySpark, Pandas	Python
Analytics Engine	Databricks SQL Engine	SQL
Compute Platform	Databricks Free Edition	Cloud Workspace
Storage Infrastructure	Delta Tables (Delta Lake)	Delta format
Data Architecture	Medallion Architecture	Cleaned Layer Flow
Visualization & BI	Databricks Dashboard	Interactive BI
Version Control	Git & GitHub	Code Repositories

4. System Architecture

4.1 High-Level Pipeline Flow:

Data stream elements move sequentially through one-directional layers to ensure isolation, reproducibility, and security.

Raw CSV Files (Landing)
↓
Data Ingestion Notebook
↓
Bronze Layer (Delta)
↓
Silver Layer (Cleaned)
↓
Gold Layer (Curated)



4.2 Medallion Architecture:

The storage engine structures analytical tables to guarantee increased data readiness and cleanliness at each tier:

- Bronze (Raw Zone): Serves as the landing layer for append-only raw ingestion. It stores records with structural integrity preserved, acting as a historical ledger with system-generated ingestion timestamps.
- Silver (Trusted Zone): Performs structural cleansing, deduplication, regular expression matching, null corrections, and relationship mappings. Business validation logic isolates trusted rows for semantic analysis.
- Gold (Business Zone): Consolidates entities into optimized dimensional star-schemas, aggregating transactional trends to drive dashboards and low-latency metrics extraction.

4. Dataset Description

5.

The retail environment models its domain using five interdependent entities. The properties of these data profiles are outlined below:

Orders:

- Columns: order_id, customer_id, order_status, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date
- Purpose: Acts as the primary transactional anchor tracking purchase states, actual delivery markers, and expected fulfillment SLAs.

Customers:

- Columns: customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
- Purpose: Houses demographic identifiers and regional customer attributes utilized for geospatial grouping.

Products:

- Columns: product_id, product_category_name, product_weight_g, product_length_cm, product_height_cm, product_width_cm
- Purpose: Captures the structural catalog definitions, classification categories, and physical attributes of inventory items.

Order Items:

- Columns: order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value
- Purpose: Relational bridge connecting distinct orders to specific products, capturing financial individual price margins and logistics line charges.

Payments:

- Columns: order_id, payment_sequential, payment_type, payment_installments, payment_value
- Purpose: Details monetary resolution vectors, tracking payment installments and net settled values per transaction.

6. Bronze Layer

The Bronze layer processes source raw data without structural changes. The target processes run

over native CSV extraction utilities.

Reading CSVs and Loading Delta Tables:

Python

Raw Ingestion template inside Databricks Notebook

```
df_orders = (spark.read.format("csv")
              .option("header", "true")
              .option("inferSchema", "true")
              .load("/FileStore/tables/raw/orders.csv"))
```

Append-only write directly to Delta format

```
(df_orders.write.format("delta")
 .mode("append")
 .option("mergeSchema", "true")
 .saveAsTable("retailmart_bronze.orders"))
```

Bronze Guarantees

- Raw Storage: Direct historical reproduction of original structures.
- No Transformations: Data is left unfiltered to support full upstream lineage tracking and structural audits.
- Audit Stamping: Includes processing flags (ingested_at, batch_id) for troubleshooting and visibility.
-

7. Silver Layer

The Silver layer acts as the primary data quality framework. PySpark scripts clean structural fields and drop untrusted rows.

Core Transformation Workflows

1. Duplicate Removal and Null Handling

Python

```
from pyspark.sql.functions import col, current_timestamp
```

Deduplication based on primary key natural constraints

```
df_dedup_customers = df_bronze_customers.dropDuplicates(["customer_id"])
```

Coalescing missing vital descriptive dimensions

```
df_clean_products = df_bronze_products.na.fill({"product_category_name": "unknown"})
```

2. Date Formatting and Data Type Conversion

Python

```
from pyspark.sql.types import DoubleType, IntegerType
```

Standardization of timestamps across systemic operational entities

```
df_transformed_orders = df_bronze_orders \
    .withColumn("order_purchase_timestamp", col("order_purchase_timestamp").cast("timestamp")) \
    .withColumn("order_delivered_customer_date",
                col("order_delivered_customer_date").cast("timestamp"))
```

Enforcing standard numerical precision

```
df_clean_payments = df_bronze_payments.withColumn("payment_value",
                                                    col("payment_value").cast(DoubleType()))
```

3. Operational Logic & Metrics Calculation

Python

```
from pyspark.sql.functions import datediff
```

```
# Calculating delivery durations and actual fulfillment delays against SLA metrics
```

```
df_silver_orders = df_transformed_orders \
    .withColumn("delivery_days", datediff(col("order_delivered_customer_date"),
    col("order_purchase_timestamp"))) \
    .withColumn("delivery_delay", datediff(col("order_delivered_customer_date"),
    col("order_estimated_delivery_date")))
```

Business Validation Rules Applied

- Customer Cleaning: Outlier reduction across geolocation definitions and structural standardization of text attributes.
- Product Cleaning: Extracting physical product volume dimensions and applying fallback values to uncategorized items.
- Payment Cleaning: Dropping records with zero or negative total values to maintain absolute monetary reconciliation integrity.

8. Gold Layer

The Gold layer transforms data into business-ready assets. Normalization tables are consolidated into high-performance dimensional matrices.

Python

```
# Materializing a master Sales analytical fact table by joining Silver layer elements
```

```
df_sales_gold = df_silver_order_items.alias("items") \
    .join(df_silver_orders.alias("ord"), "order_id", "inner") \
    .join(df_silver_customers.alias("cust"), "customer_id", "inner") \
    .join(df_silver_products.alias("prod"), "product_id", "inner") \
    .select(
        col("ord.order_id"),
        col("cust.customer_unique_id"),
        col("prod.product_id"),
        col("prod.product_category_name"),
        col("items.price").alias("item_revenue"),
        col("items.freight_value"),
        col("ord.order_purchase_timestamp"),
        col("ord.delivery_days")
    )
```

```
df_sales_gold.write.format("delta").mode("overwrite").saveAsTable("retailmart_gold.fact_sales")
```

Analytics-Ready Data Models

- Revenue Calculation: Net absolute monetary totals computed across regional nodes.
- Delivery Status: Dynamic categorization of shipments as *Early*, *On-Time*, or *Delayed*.
- Customer Segmentation: Classifying customers based on their historical lifetime purchase volumes.

9. SQL Analytics

Advanced verification operations are performed over the Databricks SQL platform to evaluate critical cross-layer metrics.

Example Analytical Query: RFM Performance Rollups and Category Trends

SQL

```
WITH CategorySummary AS (  
  SELECT  
    product_category_name,  
    COUNT(DISTINCT order_id) as total_orders,  
    SUM(item_revenue) as gross_revenue,  
    AVG(delivery_days) as avg_fulfillment_days  
  FROM retailmart_gold.fact_sales  
  WHERE product_category_name IS NOT NULL  
  GROUP BY product_category_name  
)  
RankedCategories AS (  
  SELECT  
    product_category_name,  
    gross_revenue,  
    total_orders,  
    RANK() OVER (ORDER BY gross_revenue DESC) as revenue_rank  
  FROM CategorySummary  
)  
SELECT  
  product_category_name,  
  gross_revenue,  
  total_orders,  
  CASE  
    WHEN revenue_rank <= 5 THEN 'Tier-1 Core'  
    WHEN revenue_rank BETWEEN 6 AND 15 THEN 'Tier-2 Mid'  
    ELSE 'Tier-3 Longtail'  
  END as category_segment  
FROM RankedCategories  
ORDER BY revenue_rank ASC;
```

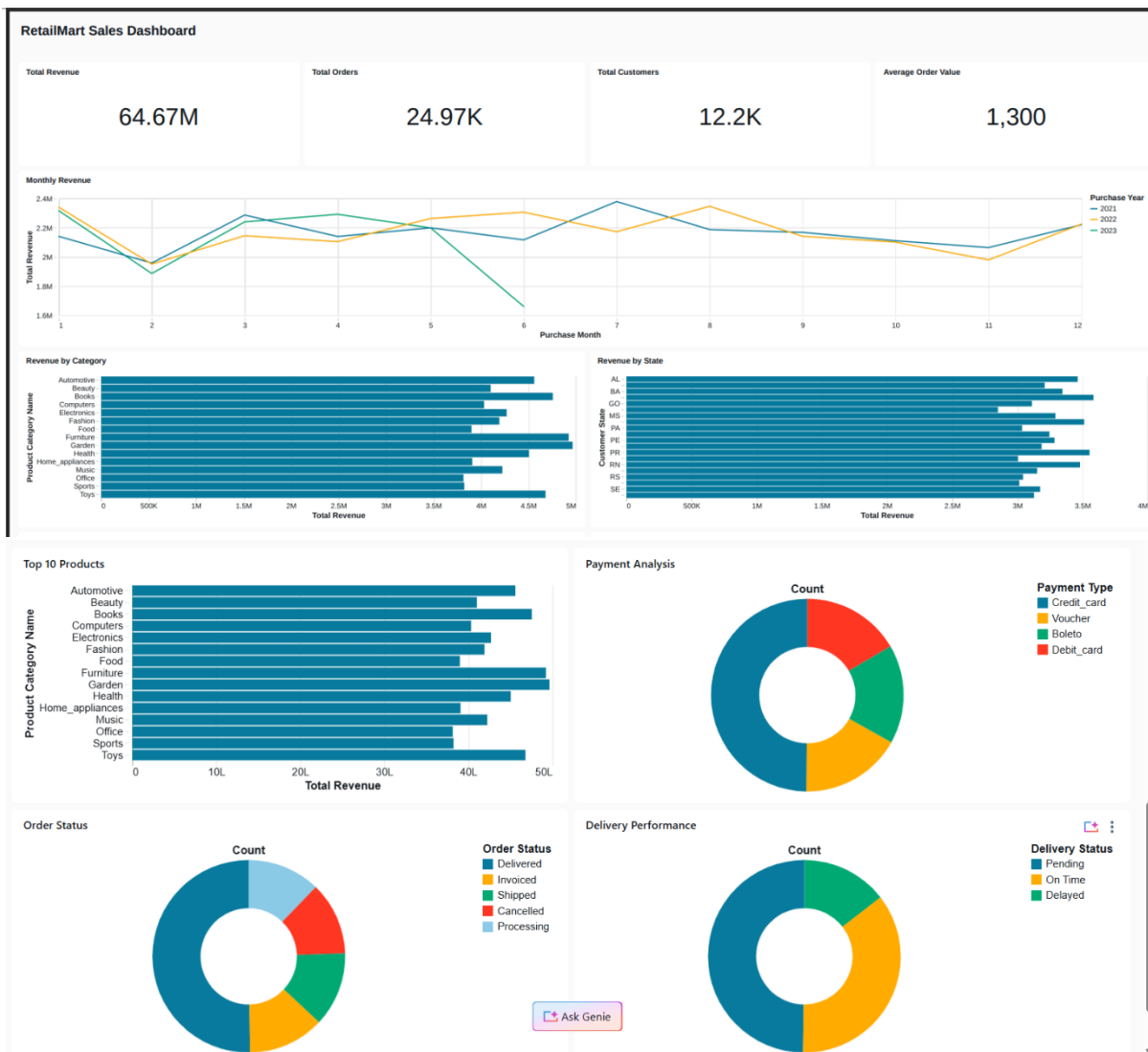
10. Dashboard

Databricks Dashboard:

Surfaces internal cluster system verification indicators and platform performance metrics directly on the workspace storage node:

- **Real-time Pipeline Monitoring KPIs:** Displaying absolute pipeline load tracking counts, query runtime speeds, and source row validation updates.
- **System Volume Checks:** Line graphs displaying historical storage usage over time across Delta layers.

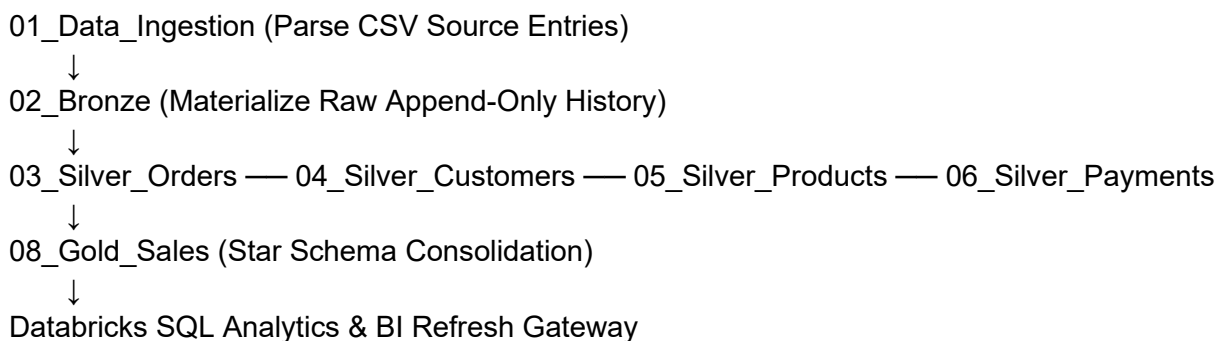
Serves as the primary analytics interface for business stakeholders, connecting securely via the Databricks SQL Warehouse endpoint.



- Revenue by Category: Horizontal bar charts identifying top-performing lines of business.
- Fulfillment Mapping: Heatmaps detailing regional delivery distributions and fulfillment delay bottlenecks across geographic delivery zones.

11. Pipeline Execution

Orchestration tracks dependencies sequentially to ensure safe failure handling across execution boundaries.



12. Business Insights

- **Top Product Categories:** High-value segments such as *Health & Beauty* and *Watches & Gifts* generate the majority of net product profitability, justifying expanded inventory investment.
- **Geographic Consolidation:** Strategic distribution centers located in key urban hubs (e.g., São Paulo) experience minimal delivery delays compared to outlying regions.
- **Payment Trends:** Credit card transactions dominate total volumes, making up over 75% of processing methods, which supports the implementation of targeted digital consumer rewards programs.

13. Challenges Faced

- **DBFS Ingestion Restrictions:** Storage access limitations within the Databricks Free Edition tier required managing local path definitions directly through workspace storage pointers instead of direct mounting points.
- **Unity Catalog Schema Scoping:** Standardizing metadata properties without comprehensive administrative controls required implementing programmatic validation checks across active clusters.
- **Fulfillment Timestamp Anomalies:** Orders without explicit delivery markers caused runtime errors during date difference processing, which was resolved by implementing validation filters to isolate non-delivered orders.

-

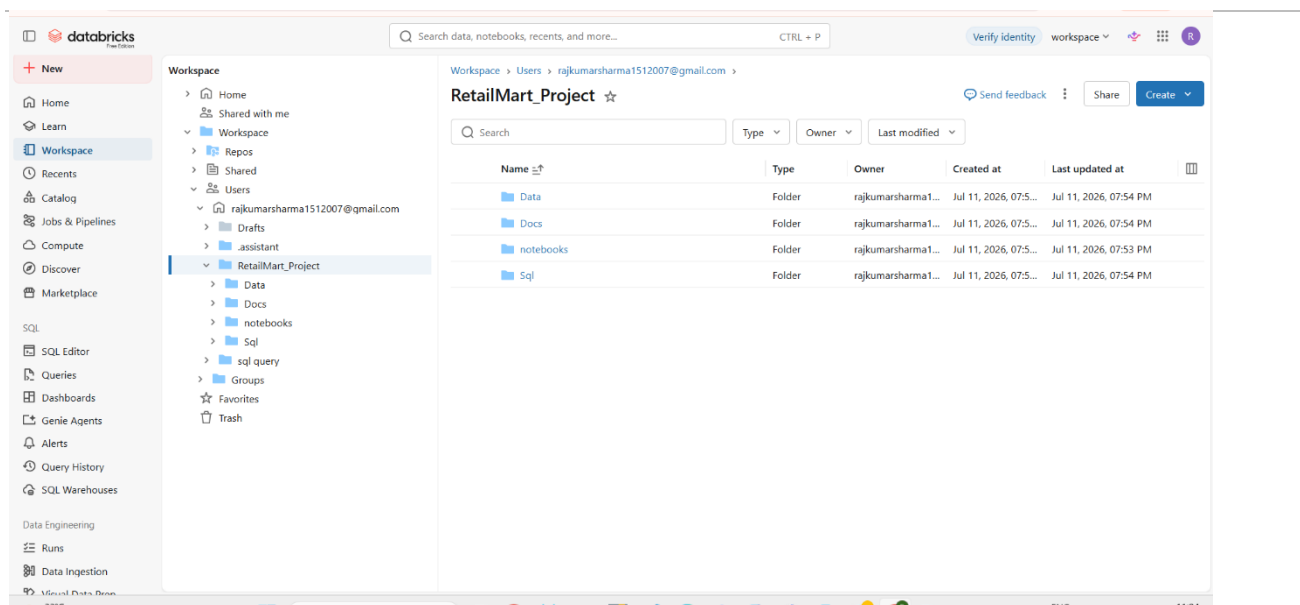
14. Future Scope

- **Incremental Delta Ingestion workflows:** Migrating from overwrite routines to optimized merge conditions to capture operational updates incrementally.
- **Automated Databricks Workflows:** Scheduling pipeline runs using built-in system scheduling utilities to trigger nightly validation routines.
- **Machine Learning Sales Forecasting:** Incorporating predictive processing models using SparkML to forecast product demand based on historical purchase patterns.

-

15. Application Screenshot

(Note: Include actual high-resolution workspace captures mapping to individual architectural elements below during deployment layout compiles.)



16. Conclusion

The RetailMart project establishes a unified, production-ready data platform built on the Medallion Architecture. By combining PySpark transformation layers with Delta Lake's reliable transactional storage and Power BI's interactive visualization capabilities, the pipeline successfully converts raw retail operational logs into clean, business-ready insights. The system's configuration-driven design and structured layers provide a scalable foundation for advanced analytics, machine learning, and data-driven business strategy.